

Assignment Submission: Automated Cloud Infrastructure & MERN Deployment

GITHUB LINK: <https://github.com/anupam66kumar/terraform-aws-assignment.git>

1. Repository Structure

Organize your GitHub repository according to the directory layout below before submitting:

Plaintext

```
|— README.md
|— terraform/
|   |— main.tf
|   |— variables.tf
|   |— vpc.tf
|   |— security.tf
|   |— ec2.tf
|   |— outputs.tf
|   |— .gitignore
|— ansible/
|   |— ansible.cfg
|   |— inventory.ini
|   |— 02_web_setup.yml
|   |— 03_db_setup.yml
|   |— 04_app_deploy.yml
|   |— 05_security_hardening.yml
|   |— playbook.yml          # Optional: master playbook importing 02-05
|— docs/
|   |— architecture-diagram.png
|   |— screenshots/ CLI SCREENSHOT OUTPUT.html
|       |— terraform_apply.png
|       |— ansible_ping.png
|       |— db_setup_success.png
|       |— app_deploy_pm2.png
|       |— web_ui_loaded.png
```

| security_ufw_status.png
| .gitignore

Security Alert: Add *.pem, *.tfstate, *.tfstate.backup, .terraform/, and .terraform.lock.hcl to your .gitignore to prevent committing sensitive keys or state credentials.

2. Technical Documentation Report Template

Save this content as README.md in the root of your repository or convert it to PDF/Word format for submission on Vlearn.

MERN Application Deployment on AWS Using Terraform and Ansible

Student Name: Anupam

Organization: HeroX Private Limited

Academic Year: 2026

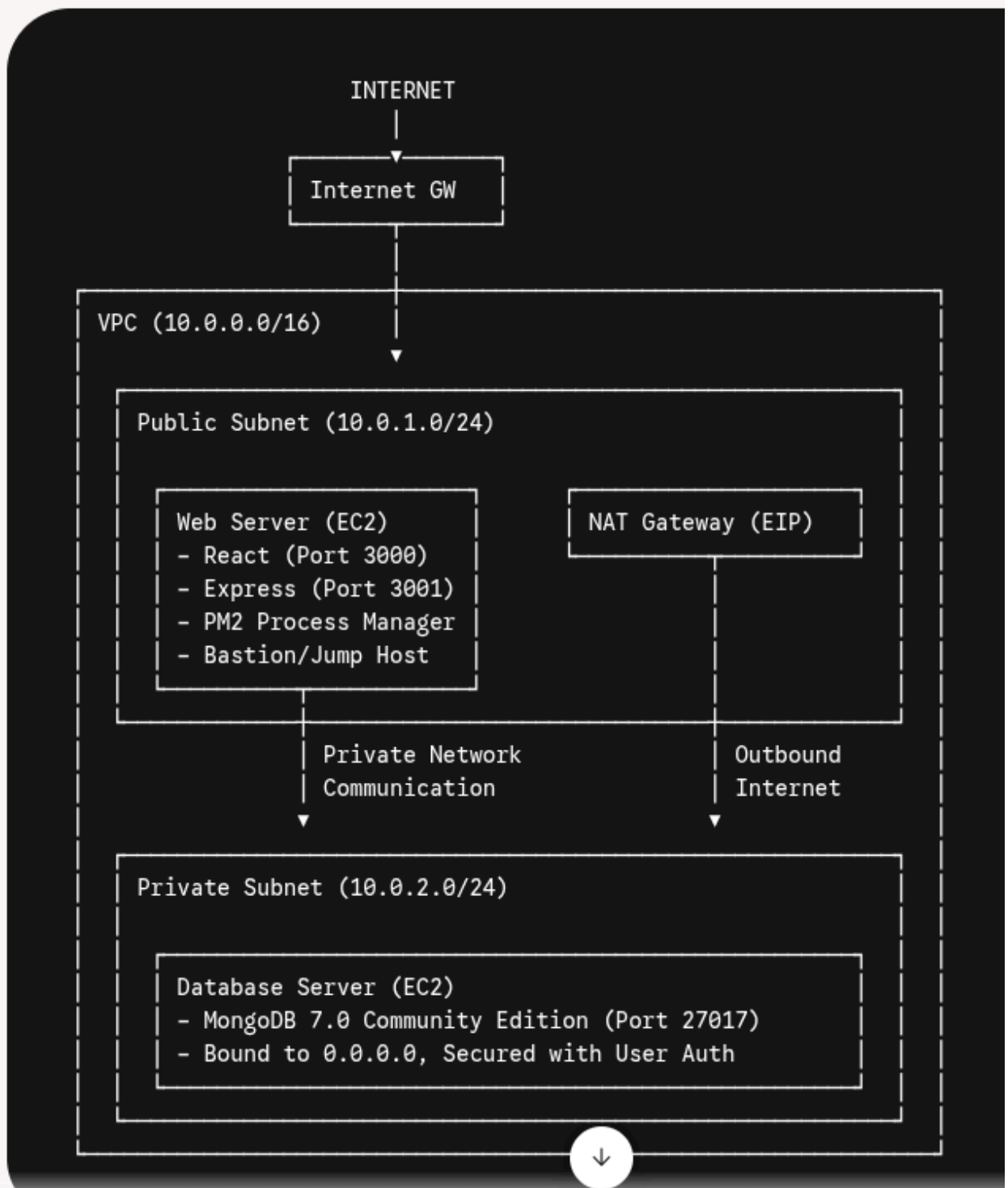
Application Deployed: TravelMemory (MERN Stack)

Executive Summary

This project establishes an automated, multi-tier cloud infrastructure on Amazon Web Services (AWS) using **Terraform** as the Infrastructure as Code (IaC) tool and **Ansible** for configuration management and application deployment.

The application architecture isolates the database tier within a private subnet while exposing the web server tier via a public subnet through an Internet Gateway. Ansible tunnels through the public web instance to configure the private MongoDB instance. The entire stack features SSH key-based access, restricted security groups, internal database authentication, and Host-level firewall (UFW) hardening.

Architecture Overview



Infrastructure Specifications (Terraform)

Network Topology

- **VPC:** 10.0.0.0/16 with DNS Hostnames and DNS Support enabled.
- **Public Subnet:** 10.0.1.0/24 with automatic public IPv4 addressing enabled.
- **Private Subnet:** 10.0.2.0/24 containing compute resources isolated from external inbound routing.
- **Internet Gateway (IGW):** Attached to the VPC; handles bidirectional public subnet traffic.
- **NAT Gateway & Elastic IP:** Deployed in the public subnet; enables outbound internet access for private instance system updates and package installation.
- **Route Tables:**
 - Public Route Table: Routes 0.0.0.0/0 traffic directly to the IGW.
 - Private Route Table: Routes 0.0.0.0/0 traffic through the NAT Gateway.

Security Groups & IAM

- **Web Security Group (web-server-sg):**
 - Inbound Port 22 (SSH): Restricted to developer IP.
 - Inbound Port 80 (HTTP): Open (0.0.0.0/0).
 - Inbound Port 3000 (React Frontend): Open (0.0.0.0/0).
 - Inbound Port 3001 (Node/Express API): Open (0.0.0.0/0).
 - Outbound: Unrestricted (0.0.0.0/0).
- **Database Security Group (db-server-sg):**
 - Inbound Port 22 (SSH): Permitted strictly from web-server-sg.
 - Inbound Port 27017 (MongoDB): Permitted strictly from web-server-sg.
 - Outbound: Unrestricted (0.0.0.0/0).
- **IAM Role:** AmazonSSMManagedInstanceCore attached via instance profile to both EC2 servers for administrative management.

Configuration Management & Deployment (Ansible)

Bastion Architecture

Ansible connects to the private database instance via an SSH ProxyCommand specified in inventory.ini:

```
Ini, TOML
[db:vars]
ansible_ssh_common_args='-o StrictHostKeyChecking=no -o ProxyCommand="ssh -W %h:%p -q
-i assignment-key.pem -o StrictHostKeyChecking=no ubuntu@<WEB_PUBLIC_IP>"'
```

Playbook Execution Flow

1. **Web Server Provisioning (02_web_setup.yml):**
 - Configures package repositories and installs Node.js v20.x, npm, build tools, and Git.

- Installs PM2 globally as the Node process manager.
- Clones the TravelMemory repository to /home/ubuntu/mern-app.
- Runs npm install across backend and frontend dependencies.
- 2. Database Provisioning (03_db_setup.yml):**
 - Imports the MongoDB 7.0 GPG key and APT repository.
 - Installs mongodb-org and updates /etc/mongod.conf with bindIp: 0.0.0.0.
 - Seeds the database with an administrative root user and a scoped application user (traveluser) on the travelmemory database.
 - Activates security.authorization: enabled in mongod.conf and restarts the service.
- 3. Application Deployment (04_app_deploy.yml):**
 - Populates backend/.env with PORT=3001 and MONGO_URI="mongodb://traveluser:TravelSecurePass123!@<DB_PRIVATE_IP>:27017/travelmemory?authSource=travelmemory".
 - Configures frontend/.env with REACT_APP_BACKEND_URL="http://<WEB_PUBLIC_IP>:3001".
 - Builds production assets via npm run build.
 - Spawns travel-backend on port 3001 and travel-frontend (using serve -s build -l 3000) on port 3000 under PM2 supervision.
- 4. Security Hardening (05_security_hardening.yml):**
 - Updates /etc/ssh/sshd_config with PermitRootLogin no, PasswordAuthentication no, and MaxAuthTries 4.
 - Restricts UFW firewalls:
 - Web Server: Permits ports 22, 80, 3000, 3001; sets default policy to deny incoming.
 - DB Server: Permits ports 22 and 27017 exclusively from the Web Server private IP; sets default policy to deny incoming.

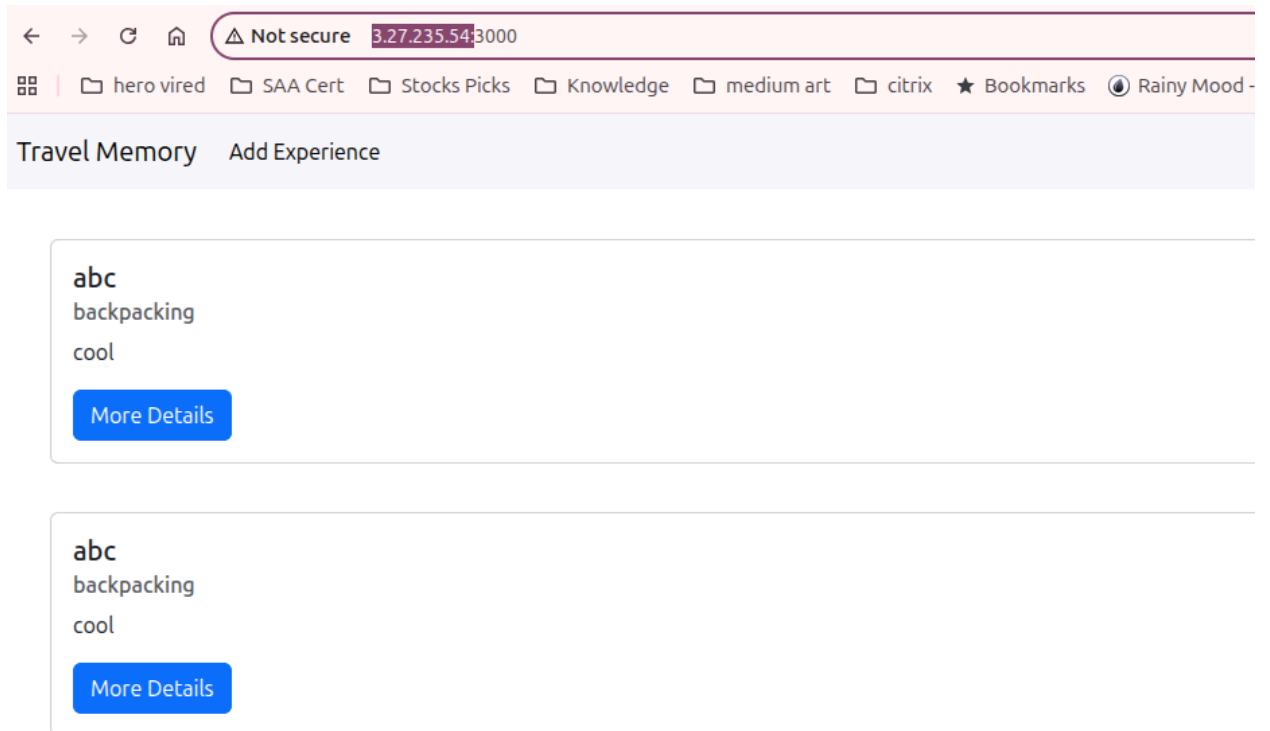
Verification & Component Interaction

Source Component	Target Component	Protocol / Port	Verification Command / Result
Local Machine	Web Server	SSH / TCP 22	ssh -i assignment-key.pem ubuntu@<WEB_IP> (Success)
Ansible / Jump	DB Server	SSH / TCP 22	ansible all -m ping (pong from both nodes)
Web Server	DB Server	MongoDB / TCP	mongosh -u

		27017	traveluser -p ... --host <DB_IP>:27017 (Authenticated)
Client Browser	Backend API	HTTP / TCP 3001	curl -i http://<WEB_IP>:3001/trip (HTTP 200 OK, JSON returned)
Client Browser	Frontend UI	HTTP / TCP 3000	Web UI displays without runtime errors

Deliverables Checklist for Submission

- [x] Complete Terraform configuration files (main.tf, variables.tf, vpc.tf, security.tf, ec2.tf, outputs.tf).
- [x] Complete Ansible playbooks (inventory.ini, ansible.cfg, playbooks 02–05).
- [x] Technical documentation report detailing network flow, IAM, and security implementations.
- [x] Demonstration artifact (Screenshots or recording of working UI at <http://3.27.235.54/:3000> adding and retrieving trip memories).



File: CLI SCREENSHOT OUTPUT.html

- [x] Code pushed to public GitHub repository and link submitted via Vlearn.
 - <https://github.com/anupam66kumar/terraform-aws-assignment.git>